



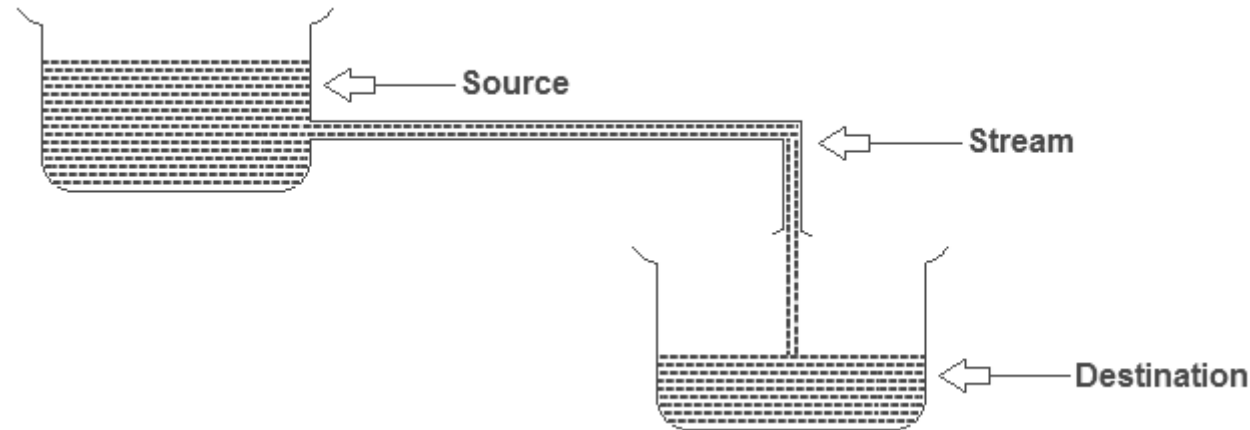
**We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech**

# I/O Stream



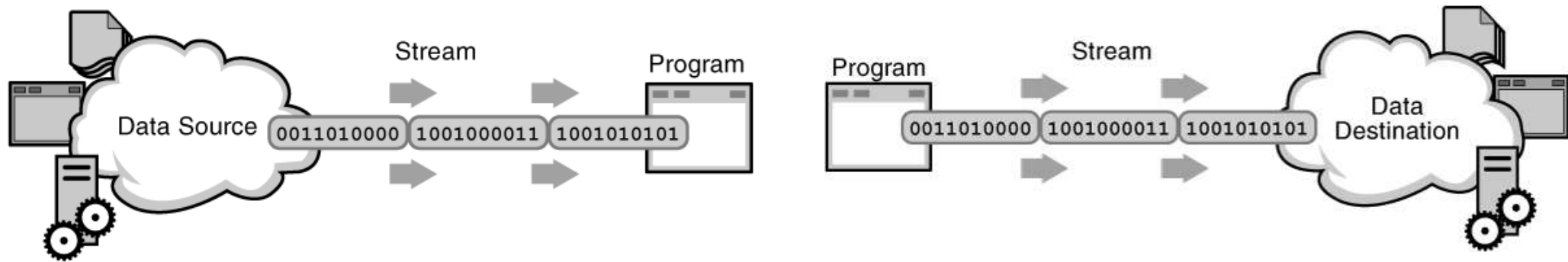
## Introduction

- **I/O Stream** concept is used in **Java** to make **faster I/O operations**.
- A stream is a **sequence of data**. It's called a stream because it is like a **stream of water** that continues to flow.



## Introduction

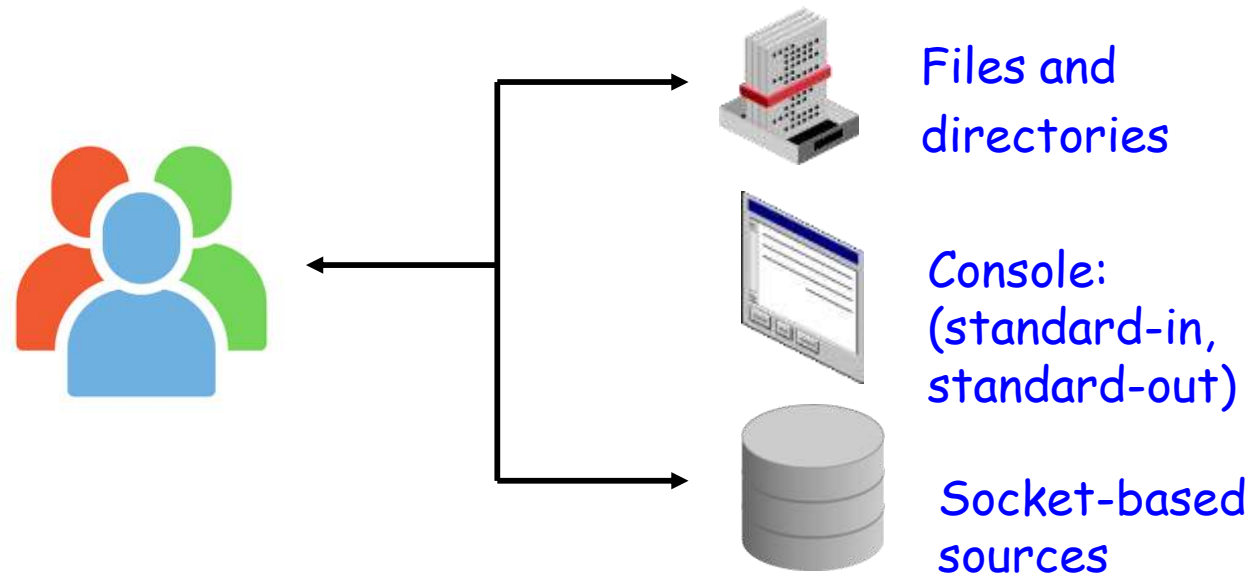
- Streams represent **input source** and **output destination**.
- A program uses an **input stream** to **read data from a source**, one item at a time.
- A program uses an **output stream** to **write data to a destination** (sink), one item at a time.





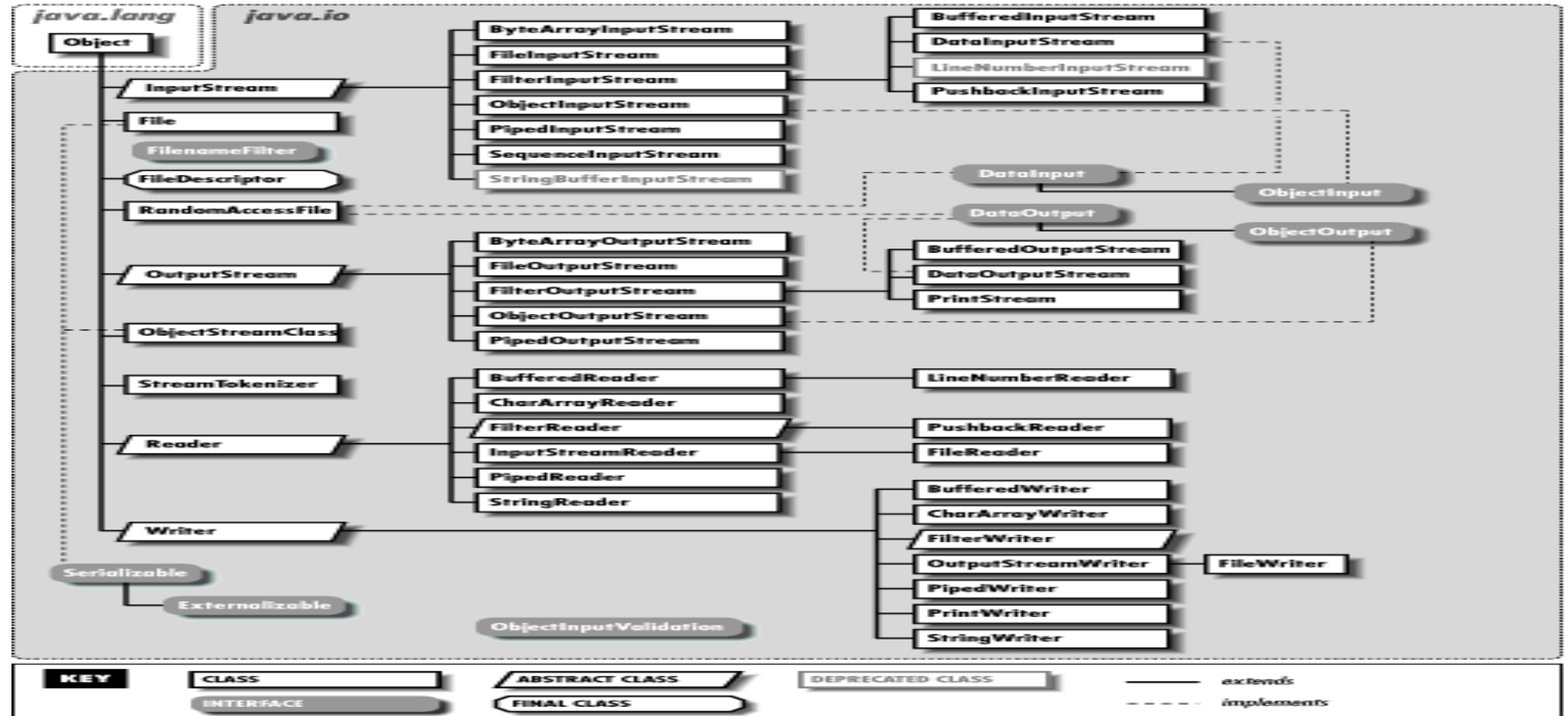
## Introduction

- I/O streams support **different kinds of data**, including simple **bytes**, **primitive data types**, **localized characters**, and **objects**. Typically, a **developer** uses input and output in **three ways**:



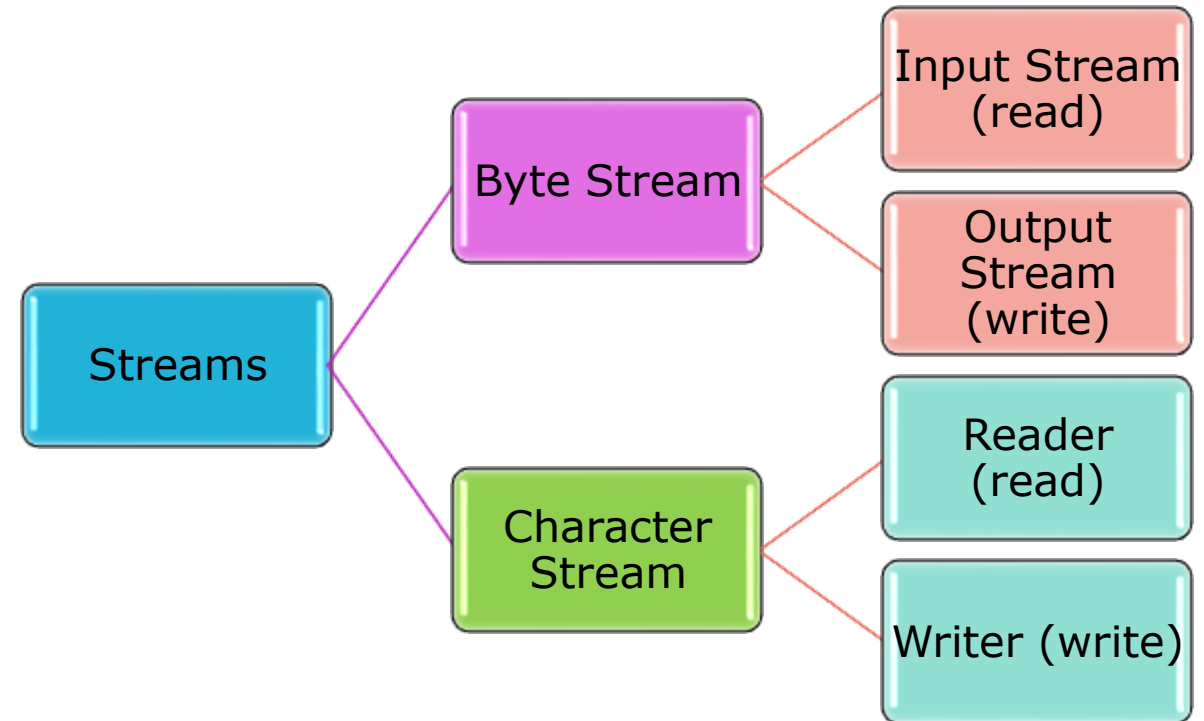
- In Java, **java.io package** contains all the **classes and methods** need to support Input and Output operations.

## Java.io package Hierarchy



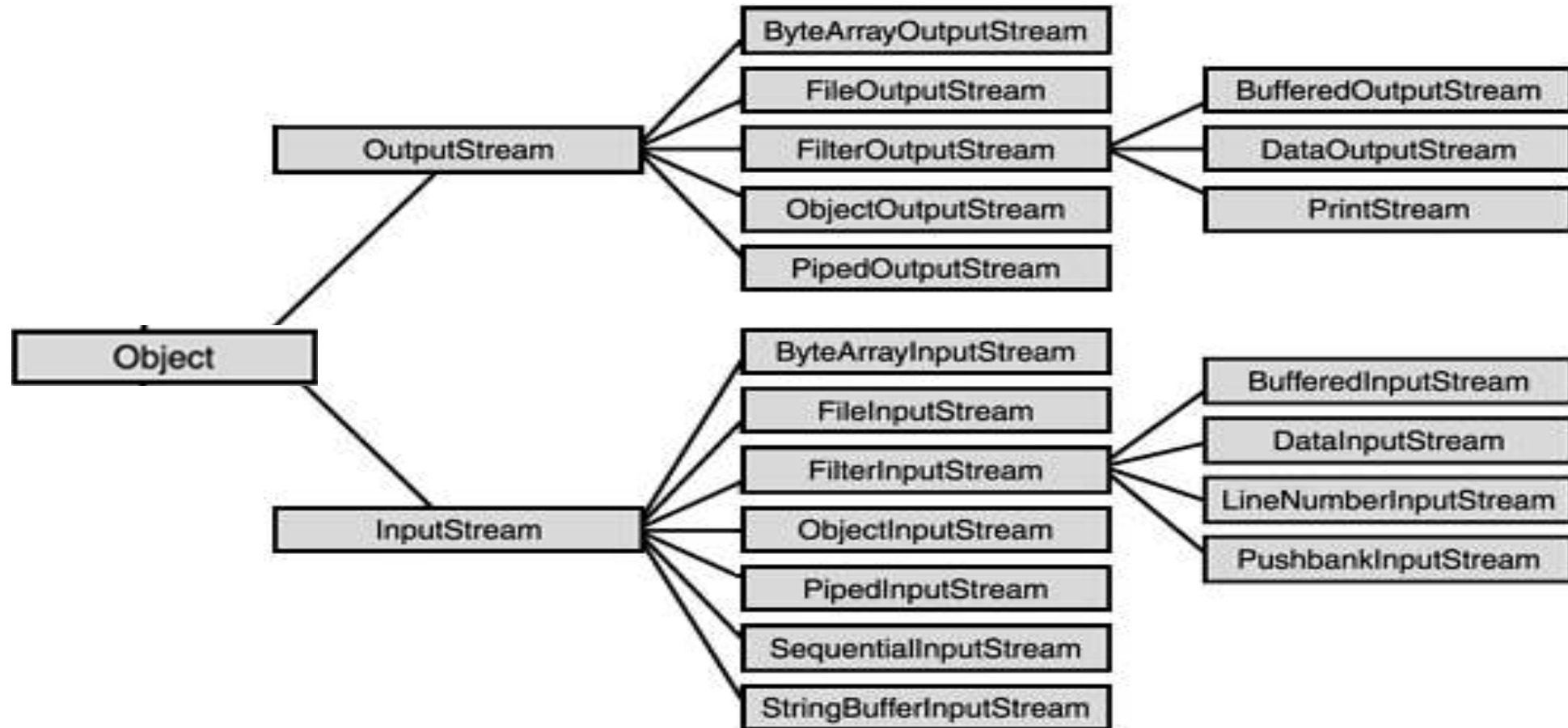
## Data within Streams

- Java technology supports two types of **streaming** data: raw **bytes** and **Unicode characters**. The term streams usually refers to **byte** and **character streams**.
- **Byte Stream**: It is used to process **data as bytes**. The bytes of data such as image files, audio files, and objects.
- **Character Stream**: It is used to process **data as characters**. It includes files and other character-based streams.



## Byte Stream Classes

- Specific subclasses provide methods **for providing** specific support for each of these **types of streams**.





## Byte Stream Classes

- Programs use **byte streams** to perform input and output of **8-bit bytes**.
- Byte oriented streams **do not use** any **encoding scheme**.

| Classes                                                 | Description                                                                                                                                                  |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>FileInputStream /<br/>FileOutputStream</i>           | Read data from or write data to a <b>file</b> on the native file system.                                                                                     |
| <i>ByteArrayInputStream /<br/>ByteArrayOutputStream</i> | Read data from or write data to a <b>byte array</b> in memory.                                                                                               |
| <i>PipedInputStream /<br/>PipedOutputStream</i>         | Implement the input and output components of a <b>pipe</b> .<br>Pipes are used to channel the output from one program (or thread) into the input of another. |
| <i>FilterInputStream /<br/>FilterOutputStream</i>       | Filtered streams which process data as it's being read or written.                                                                                           |

## Byte Stream Classes

| Classes                                               | Description                                                                                                                                                               |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>DataInputStream /<br/>DataOutputStream</i>         | Read or write primitive Java data types in a machine-independent format.                                                                                                  |
| <i>BufferedInputStream /<br/>BufferedOutputStream</i> | Buffer data while reading or writing, thereby reducing the number of accesses required on the original data source. Buffer data while reading and writing to speed it up. |
| <i>SequenceInputStream</i>                            | Concatenate multiple input streams into one input stream.                                                                                                                 |
| <i>StringBufferInputStream</i>                        | Allow programs to read from a StringBuffer as if it were an input stream.                                                                                                 |
| PushbackInputStream                                   | An input stream with a one-byte pushback buffer.                                                                                                                          |
| <b>LineNumberInputStream</b>                          | <b>Deprecated</b>                                                                                                                                                         |

## InputStream and Output Stream Methods

- **InputStream:**
- The three basic **read methods** are,

```
int read()  
int read(byte[] buffer)  
int read(byte[] buffer, int offset, int length)
```

- Other methods are,

```
void close();  
int available();  
long skip(long n);
```

- **More Details:** <https://docs.oracle.com/javase/7/docs/api/java/io/InputStream.html>

## InputStream and Output Stream Methods

- **OutputStream:**

- The three basic **write methods** are,

```
void write(int c)
void write(byte[] buffer)
void write(byte[] buffer, int offset, int length)
```

- **Other methods** are,

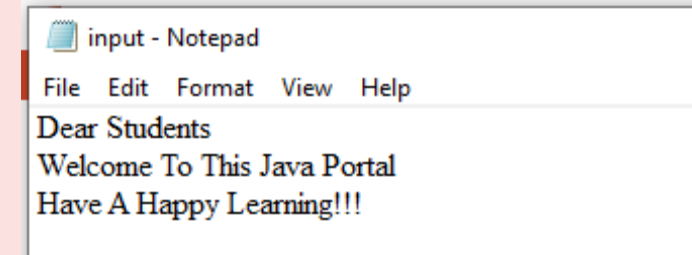
```
void close();
void flush();
```

- **More Details:** <https://docs.oracle.com/javase/7/docs/api/java/io/OutputStream.html>

# FileInputStream: Example

```
/** This example demonstrate the FileInputStream */
public class FileInputStreamDemo {
    public static void main(String args[]) {
        try {
            FileInputStream input = new FileInputStream("input.txt");
            System.out.println("Data in the file: ");
            int i = input.read(); // Reads the first byte
            while(i != -1) {
                System.out.print((char)i);
                i = input.read(); // Reads next byte from the file
            }
            input.close(); //close the file stream
        }
        catch(Exception e) {
            System.out.println(e); //Exception details
        }
    }
}
```

**input.txt**



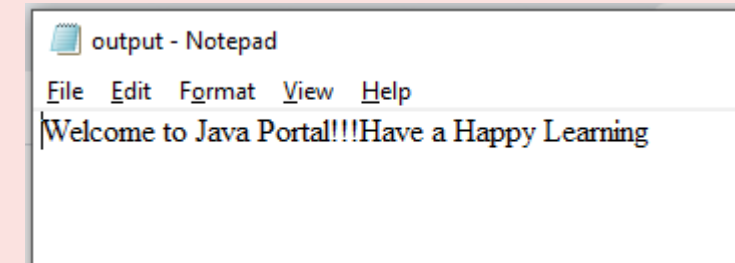


# FileOutputStream: Example

```
/** This example demonstrate the FileOutputStream */
import java.io.FileOutputStream;

public class FileOutputStreamDemo {
    public static void main(String[] args) {
        String data = "Welcome to Java Portal!!!Have a Happy Learning";
        try {
            FileOutputStream output = new FileOutputStream("output.txt");
            byte[] array = data.getBytes();
            // Writes byte to the file
            output.write(array);
            output.close();
        }
        catch(Exception e) {
            System.out.println(e); //Exception details
        }
    }
}
```

**output.txt**



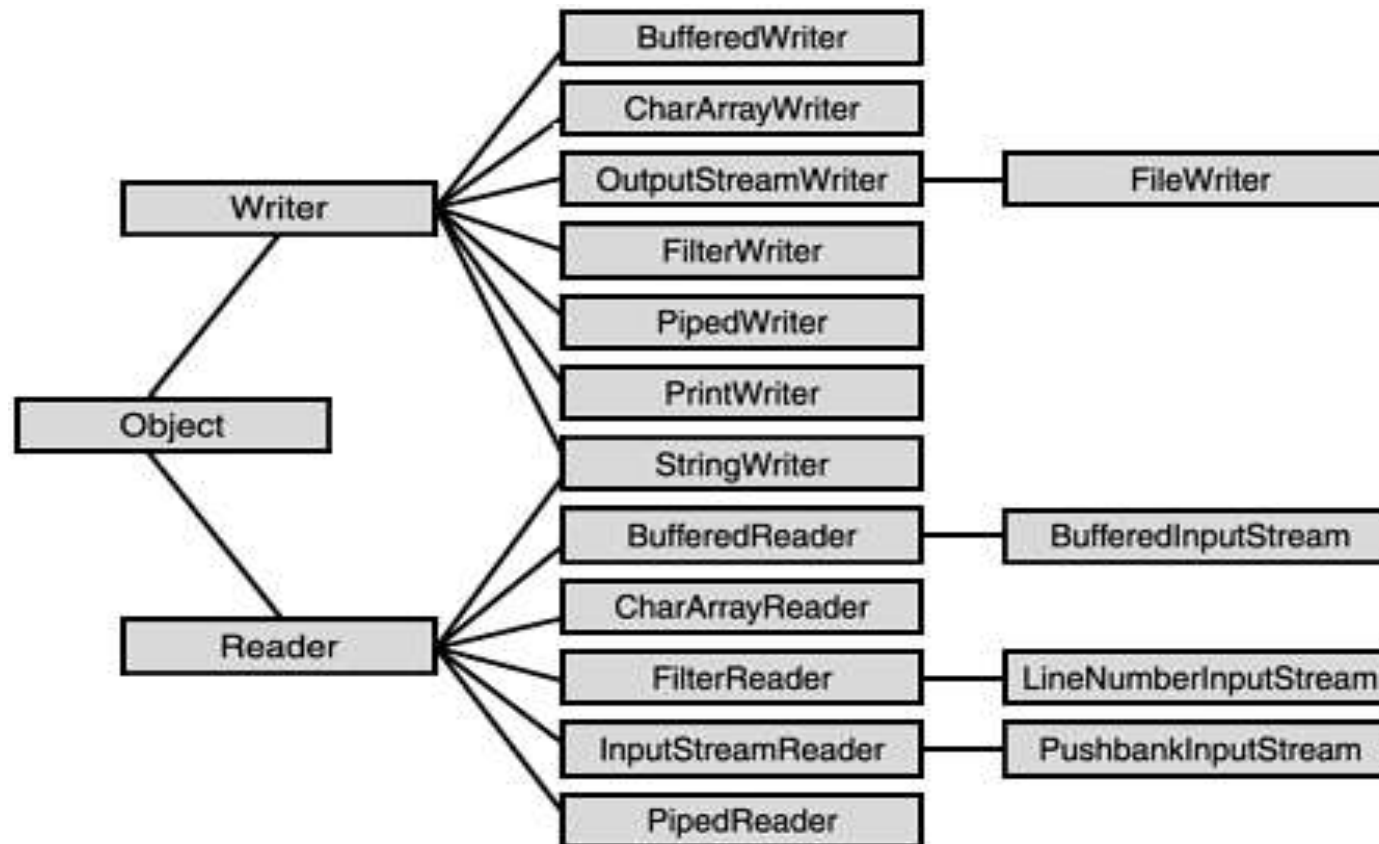
# FileInputStream and FileOutputStream : Example

/\*\*This example demonstrate the FileInputStream and FileOutputStream Classes\*/

```
class ByteIOStream{  
    public static void main(String[] args) {  
        byte[] b = new byte[128];  
        try (FileInputStream fis = new FileInputStream ("input.txt");  
            FileOutputStream fos = new FileOutputStream ("output.txt") ){  
            System.out.println ("Bytes available: " + fis.available());  
            int count = 0; int read = 0;  
            while ((read = fis.read(b)) != -1) {  
                fos.write(b);  
                count += read;  
            }  
            System.out.println ("Total Count: " + count);  
        }catch (FileNotFoundException f){  
            System.out.println ("File not found: " + f);  
        }catch (IOException e) {  
            System.out.println ("IOException: " + e);  
        }  
    }  
}
```

## Character Stream Classes

- Specific subclasses provide methods **for providing** a specific support for each of these **types of streams**.



## Character Stream Classes

- A character stream access the file **character by character**.
- Character stream can **support** all types of **character sets** ASCII, Unicode, UTF-8, UTF-16 etc.

| Classes                                      | Description                                                              |
|----------------------------------------------|--------------------------------------------------------------------------|
| <i>FileReader /<br/>FileWriter</i>           | Read data from or write data to a <b>file</b> on the native file system. |
| <i>CharArrayReader /<br/>CharArrayWriter</i> | Read data from or write data in to a <b>array</b>                        |
| <i>PipedReader /<br/>PipedWriter</i>         | Implement the input and output components of a <b>pipe</b> .             |
| <i>FilterReader /<br/>FilterWriter</i>       | Filtered streams which process data as it's being read or written.       |

## Character Stream Classes

| Classes                                           | Description                                                 |
|---------------------------------------------------|-------------------------------------------------------------|
| <i>InputStreamReader /<br/>OutputStreamWriter</i> | Used for Translates bytes to character / Character to bytes |
| <i>BufferedReader /<br/>BufferedWriter</i>        | Buffer data while reading and writing to speed it up.       |
| <i>StringReader / StringWriter</i>                | Allow programs to read and write as a string.               |
| PrintStream                                       | An output stream with convenient printing methods.          |
| <b>LineNumberReader</b>                           | <b>Deprecated</b>                                           |

# Reader and Writer Methods

## Reader:

- The three basic **read methods** are,

```
int read()  
int read(char[] buffer)  
int read(char[] buffer, int offset, int length)
```

- **Other methods** are,

```
void close()                boolean ready()  
long skip(long n)           boolean markSupported()  
void mark(int readAheadLimit)  
void reset()
```

**More Details:** <https://docs.oracle.com/javase/7/docs/api/java/io/Reader.html>

## Reader and Writer Methods

### Writer:

- The basic **write methods** are,

```
void write(int c)
void write(char[] cbuf)
void write(char[] cbuf, int offset, int length)
void write(String string)
void write(String string, int offset, int length)
```

- **Other methods** are,

```
void close();
void flush();
```

**More Details:** <https://docs.oracle.com/javase/7/docs/api/java/io/Writer.html>



# FileReader and FileWriter : Example

/\*\*This example demonstrate the FileReader and FileWriter Classes using command line arguments\*/

```
class ByteIOStream{
    public static void main(String[] args) {
        char[] b = new char[128];
        try (FileReader fr = new FileReader(args[0]);
            FileWriter fw = new FileWriter(args[1])){
            int count = 0; int read = 0;
            while ((read = fr.read(b)) != -1) {
                fw.write(b);
                count += read;
            }
            System.out.println ("Total Count: " + count+ " characters.");
        }catch (FileNotFoundException f){
            System.out.println("File " + args[0] + " not found.");
        }catch (IOException e) {
            System.out.println ("IOException: " + e);
        }
    }
}
```

## I/O Stream Chaining

- A program **rarely** uses a **single stream** object. Instead, it **links** a series of streams to process the data. A file stream is **buffered for efficiency** and then converted into **data (Java primitives) items**.



- Data is written, then **buffered**, and finally **written** to a file.



## I/O Stream Chaining: Example

```
/**This example demonstrate I/O Stream Chaining using command line arguments*/
class BufferedStream{
    public static void main(String[] args) {
        try (BufferedReader bufInput = new BufferedReader(new FileReader(args[0]));
            BufferedWriter bufOutput = new BufferedWriter(new FileWriter(args[1]))) {
            String line = "";
            while ((line = bufInput.readLine()) != null) {
                bufOutput.write(line);
                bufOutput.newLine();
            }
        } catch (FileNotFoundException f) {
            System.out.println("File not found: " + f);
        } catch (IOException e) {
            System.out.println("Exception: " + e);
        }
    }
}
```

## Console I/O

- The **System** class in the **java.lang** package has **three static instance fields**: **out**, **in**, and **err**.
- In Java, **3 streams** are created **automatically**. All these streams are attached with the **console**.

### 1) **System.out**: standard output stream

- It is a static instance of a **PrintStream object** that enables you to write to standard output.

### 2) **System.in**: standard input stream

- It is a static instance of an **InputStream object** that enables to read from standard input.

### 3) **System.err**: standard error stream

- It is a static instance of a **PrintStream object** that enables you to write to standard error.

# Console I/O

## Writing to Standard Output

- The **println** and **print** methods are part of the **java.io.PrintStream** class.
- The **println** methods print the argument **with newline character** (`'\n'`).
- The **print** methods print the argument **without a newline character**.
- The **print** and **println** methods are **overloaded** for **most primitive types** (boolean, char, int, long, float, and double) and for `char[]`, `Object`, and `String`.
- The **print(Object)** and **println(Object)** methods call the **toString** method on the argument.

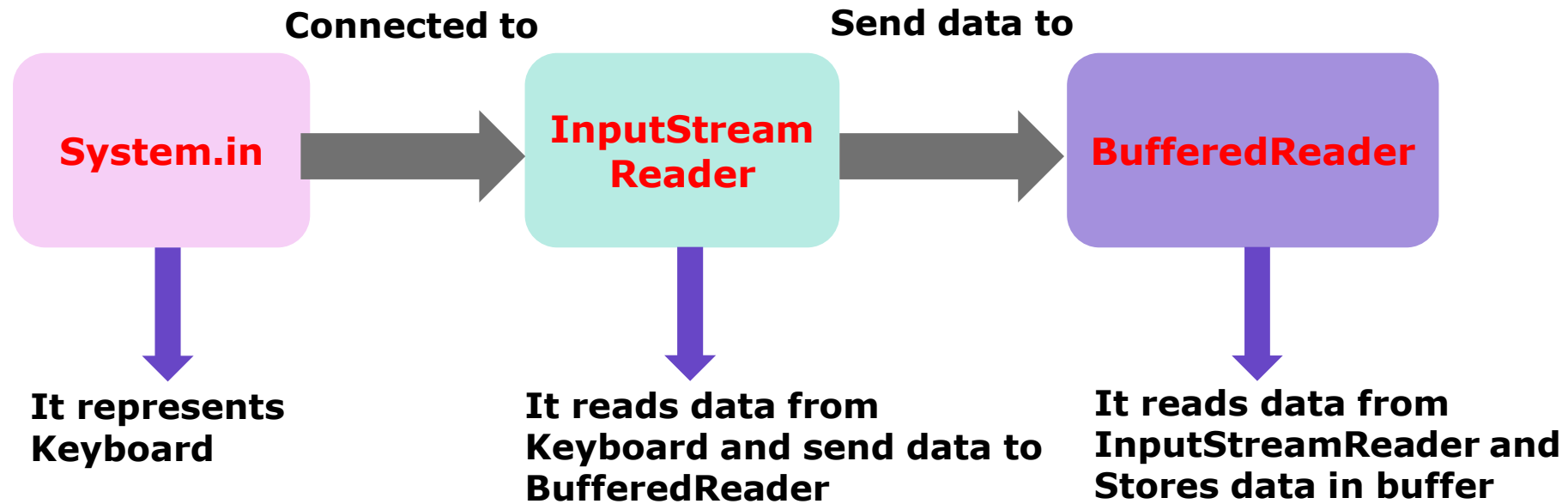
## Console I/O

### Read User Input from console

- **InputStreamReader** class can be used to **read data from console / keyboard**.
- It performs **two tasks**:
  - 1) connects to input stream of **keyboard**
  - 2) converts the **byte-oriented stream** into **character-oriented stream**
- **BufferedReader** Class is used to read the text from a character based input stream. It can be used to read data **line by line** using **readLine()** method.
- It makes the **performance fast**.

## Console I/O

Read User Input from console



## I/O Stream Chaining: Example

/\*\*This example demonstrate read user input from console using InputStreamReader and BufferedReader class\*/

```
class BufferedStream{  
    public static void main(String[] args) {  
        /*Accepting Different type of Input(integer,float,double,short,  
        long, byte, char, string, boolean) values from Keyboard*/  
        Boolean bul=false;  
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));  
        System.out.println("Enter a string:");  
        String str=br.readLine();  
        System.out.println("Enter a integer:");  
        int n=Integer.parseInt(br.readLine());  
        System.out.println("Enter float value:");  
        Float ft=Float.parseFloat(br.readLine());  
        System.out.println("Enter short value:");  
        Short sht=Short.parseShort(br.readLine());  
        System.out.println("Enter a Double value:");  
        Double dub=Double.parseDouble(br.readLine());  
    }  
}
```



## I/O Stream Chaining: Example

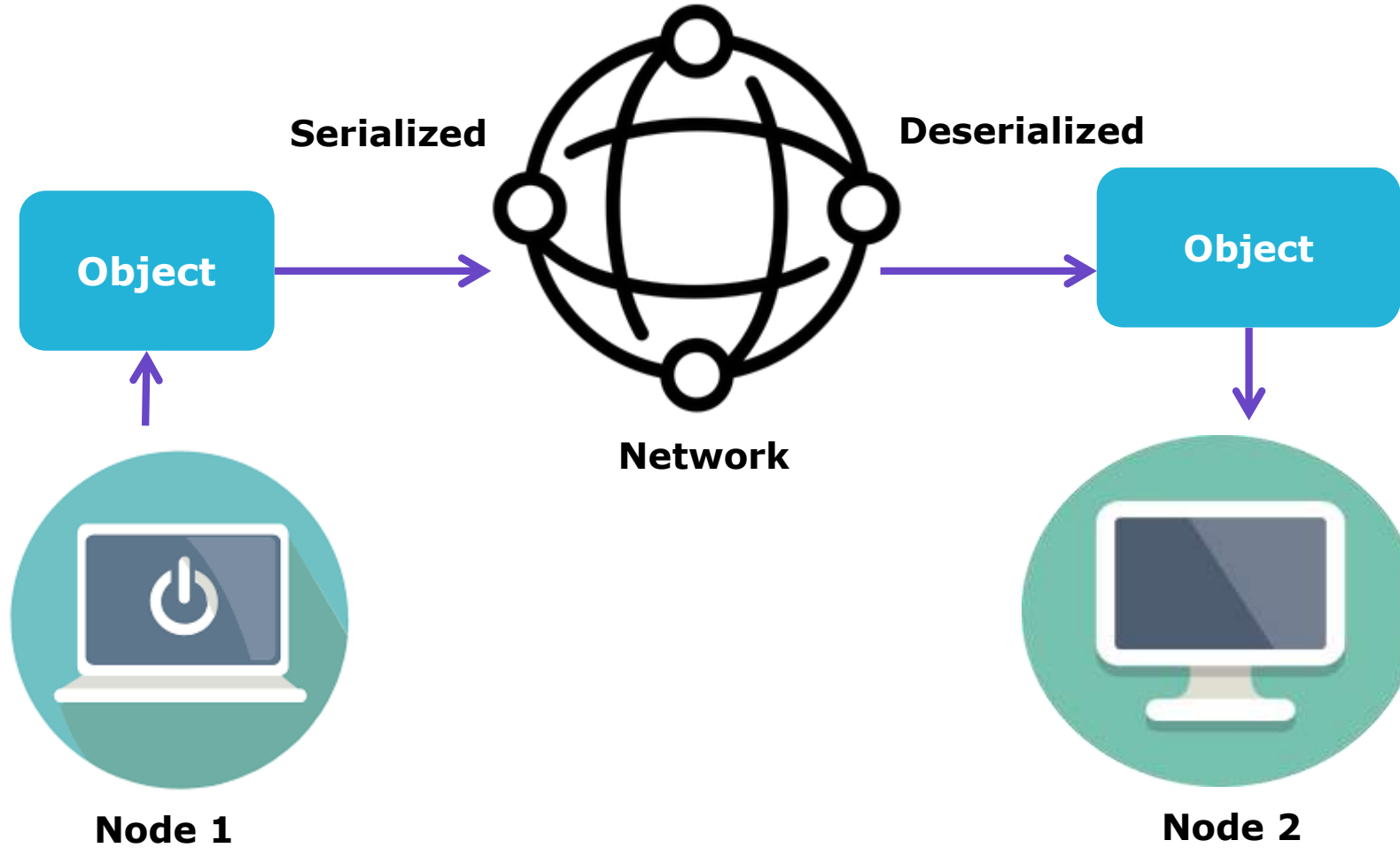
```
System.out.println("Enter a long value:");
long log=Long.parseLong(br.readLine());
System.out.println("Enter a Byte value:");
Byte bit=Byte.parseByte(br.readLine());
System.out.println("Enter a character:");
char ch=(char)br.read();
//Displaying values on console
System.out.println("Entered Values are :");
System.out.println("Integer value is: "+n);
System.out.println("Float value is: "+ft);
System.out.println("Short value is: "+sht);
System.out.println("Double value is: "+dub);
System.out.println("Long value is: "+log);
System.out.println("Byte value is: "+bit);
System.out.println("Character value is: "+ch);
System.out.println("String value is: "+str);
System.out.println("Boolean value is: "+bul);
```

```
} }
```

# Serialization and Deserialization

- **Persistence:** Saving data to some type of **permanent storage**.
- In general, a **non-persisted object** exists only as long as the **Java Virtual Machine is running**. An **object** that is persistent-capable can be stored on disk (or any other storage device), or sent to another machine to be stored there.
- **In Java** to achieve the **persistence** with the help of **serialization and deserialization** concepts.
- **Serialization** is a process to **convert objects** into a **writable byte stream**. Once converted into a byte-stream, these objects can be written to a **file or any other storage device** or **sent to another machine** to be stored there.
- **Deserialization** is the process of converting the **serialized** form of an **object back into a copy of the object**.

## Serialization and Deserialization



## Serialization and Deserialization

- A Java object is **serializable** if its **class** or any of its **superclasses** implements either the **java.io.Serializable** interface or its subinterface, **java.io.Externalizable**.
- The **ObjectOutputStream** class contains **writeObject()** method for **serializing an Object**.

**public final void writeObject(Object obj) throws IOException**

- The **ObjectInputStream** class contains **readObject()** method **deserializing an object**.

**public final Object readObject() throws IOException, ClassNotFoundException**

## Serialization and Deserialization

### Serial Version UID

- During **serialization**, a version number, **serialVersionUID**, is used to **associate the serialized output** with the class used in the serialization process.
- After **deserialization**, the **serialVersionUID** is checked to **verify** that the classes loaded are **compatible** with the **object** being **deserialized**.
- A serializable class can declare its **own serialVersionUID** by **explicitly** declaring a field named **serialVersionUID** as a **static final** and of type **long**:

**private static long serialVersionUID = 42L;**

## Serialization and Deserialization : Example

```
/**This example demonstrate the serialization and deserialization concepts*/
class Employee implements java.io.Serializable {
    public int empId;
    public String empName;

    // Parameterized Constructor
    public Employee(int id, String name) {
        this.empId = id;
        this.empName = name;
    }
}
```

## Serialization and Deserialization : Example

```
class Serialization {  
    public static void main(String[] args) {  
        Employee object = new Employee(1, "Ram");  
        String filename = "file.ser";  
        // Serialization  
        try{  
            //Saving of object in a file  
            FileOutputStream file = new FileOutputStream(filename);  
            ObjectOutputStream out = new ObjectOutputStream(file);  
            // Method for serialization of object  
            out.writeObject(object);  
            out.close();  
            file.close();  
            System.out.println("Object has been serialized");  
        }  
        catch(IOException ex) {  
            System.out.println("IOException is caught");  
        }  
    }  
}
```

## Serialization and Deserialization : Example

```
Employee object1 = null;
// Deserialization
try{ // Reading the object from a file
    FileInputStream file = new FileInputStream(filename);
    ObjectInputStream in = new ObjectInputStream(file);
    // Method for deserialization of object
    object1 = (Employee)in.readObject();
    in.close(); file.close();
    System.out.println("Object has been deserialized ");
System.out.println("Employee Id = " + object1.empId + "Employee Name = " + object1.empName);
}
catch(IOException ex){
    System.out.println("IOException is caught");
}
catch(ClassNotFoundException ex) {
    System.out.println("ClassNotFoundException is caught");
}
}
```



## I/O Stream

# Serialization and Deserialization

### Transient Keyword

- Java **transient** keyword is used in **serialization**.
- Fields that are marked as **transient** can not be part of the serialization and deserialization.

### Example:

- In Employee class, it has three data members empId, empName and empAge. If you serialize the object, all the values will be serialized but I don't want to serialize one value, **e.g. empAge** then we can declare the **empAge data member as transient**.

## Serialization and Deserialization

### Points to Remember

- If a **parent class** has implemented **Serializable interface** then child class doesn't need to implement it but vice-versa is not true.
- **Only non-static data members** are saved via Serialization process.
- **Static data members** and **transient** data members are **not saved** via Serialization process.
- **Constructor of object** is never called when an object is deserialized.

## Serialization and Deserialization : Example

```
/**This example demonstrate the serialization and deserialization concepts with transient and static members*/  
class Employee implements java.io.Serializable {  
    private static final long serialVersionUID =129348938L;  
    transient int empId;  
    static int deptId;  
    String empName;  
    int empAge;  
  
    // Default constructor  
    public Employee(String name, int age, int id, int deptid) {  
        empName = name;  
        empAge = age;  
        empId = id;  
        deptId = deptid;  
    }  
}
```

## Serialization and Deserialization : Example

```
class Serialization {  
    public static void main(String[] args) {  
        Employee object = new Employee("Ram", 29, 2, 1000);  
        String filename = "Ram.txt";  
        try { // Serialization  
            FileOutputStream file = new FileOutputStream (filename);  
            ObjectOutputStream out = new ObjectOutputStream (file);  
            out.writeObject(object); // Method for serialization of object  
            out.close();          file.close();  
            System.out.println("Object has been serialized\n"+ "Data before Deserialization.");  
            printdata(object);  
            object.deptId = 2000; // value of static variable changed  
            object.empAge=100;      // value of non static variable changed  
        }  
        catch (IOException ex) {  
            System.out.println("IOException is caught");  
        }  
        object = null;  
    }  
}
```

## Serialization and Deserialization : Example

```
try { // Deserialization
    // Reading the object from a file
    FileInputStream file = new FileInputStream(filename);
    ObjectInputStream in = new ObjectInputStream(file);
    // Method for deserialization of object
    object= (Employee)in.readObject();
    in.close();
    file.close();

    System.out.println("Object has been deserialized\n"+ "Data after Deserialization.");
    printdata(object);
}
catch (IOException ex) {
    System.out.println("IOException is caught");
}
catch (ClassNotFoundException ex) {
    System.out.println("ClassNotFoundException" + " is caught");
}
} }
```

## Quiz



**1. In java, how many streams are created for us automatically attached with console?**

a) 2

b) 4

c) 1

d) 3

e) None of the above

d) 3

# Quiz



2. Which of these class is not a member class of java.io package?

a) File

b) StringReader

c) Writer

d) String

e) None of the above

d) String

# Quiz



3. Which of these class is used to read characters in a file?

a) FileReader

b) FileWriter

c) FileInputStream

d) InputStreamReader

e) File

a) FileReader



# Quiz



4. Which of these is a method to clear all the data present in output buffers?

a) clear()

b) flush()

c) fflush()

d) close()

e) cls()

b) flush()

## Quiz



5. Which of these class can be used to implement the input stream that uses a character array as the source?

a) `BufferedReader`

b) `FileReader`

c) `CharArrayReader`

d) `FileArrayReader`

e) `ByteReader`

c) `CharArrayReader`

## Quiz



6. Which streams are used to perform input and output of 8-bit bytes?

a) Character

b) Byte

c) Bit

d) Double

e) String

b) Byte

# Quiz



7. Which one is used to output the error data produced by the user's program

a) System.in

b) System.err

c) System.out

d) System.io

b) System.err

## Quiz



8. Which of these classes defined in java.io and used for file handling are abstract?

- a) InputStream
- b) PrintStream
- c) Reader
- d) FileInputStream

a) Only a

b) Only c

c) Both a & c

d) Both b & d

e) None of these Above

c) Both a & c

## Quiz



9. Which of these is a method of `ObjectInput` interface used to deserialize an object from a stream?

a) `int read()`

b) `void close()`

c) `Object readObject()`

d) None of these above

c) `Object readObject()`

# THANK YOU